Closure, Decision, Equivalence, and Minimization

Zhengfeng Ji, *Department of Computer Science and Technology, Tsinghua University*

1 Closure Properties (cont.)

1. Union
2. Closure (star)
3. Concatenation
4. Intersection
5. Complement
6. Difference
7. Reversal
8. Homomorphism
9. Inverse homomorphism

1.1 Homomorphism

- A string homomorphism is a function on strings that works by substituting a particular string for each symbol.

  Example: $h(0) = 000$, $h(1) = 111$, and $h(w) = h(a_1)\cdots h(a_n)$.

  $h : \Sigma \to \Gamma^*$

  Proof. Induction using RE in the textbook.

  It is also easy to prove using automata.

1.2 Inverse homomorphism

- Definition

  h induces a map $\Sigma^* \to \Gamma^*$.

  $h^{-1}(L)$ is the set of ***preimages*** whose homomorphism is in L.

> Example (4.15). L is $(00 + 1)^*$, $h(a) = 01$, $h(b) = 10$. Show that $h^{-1}(L)$ is $(ba)^*$. Equivalently, $h(w) \in L$ if and only if w is of the form $baba\cdots ba$.

Proof. (If) Easy.

(Only-if) Consider the four cases if w starts with a, ends with b, has aa as a substring, or has bb as a substring. These will lead to a $0, 1$ string with a single 0 and should be excluded.

So the string has to start with b and end with a and is alternating between a and b.

□

- Inverse homomorphism has the closure property

> **Theorem**. If L is an RE, so is $h^{-1}(L)$.

Not hard—Fig 4.6 of the textbook.

Proof. Let $A = (Q, T, \delta, q_0, F)$ be the DFA for language L. Define $B = (Q, \Sigma, \gamma, q_0, F)$ where $\gamma(q, a) = \hat{\delta}(q, h(a))$.

Prove by induction on $|w|$ that $\hat{\gamma}(q, w) = \hat{\delta}(q, h(w))$.

□

1.3 Other Closure Properties

- Prefix, suffix, substring!

  $\text{Prefix}(L) = \{u \mid \text{ There is } v \in \Sigma^* \text{ such that } uv \in L\}$.

- One of your homework problems

  NOTE (Mark any state that can reach an accepting state as an accepting state).

1.4 Example

- Prove that $\{a^i b^j c^k \mid i, j, k \geq 0, j = k \text{ if } i = 1\}$ is not regular.
- The pumping lemma won't work!
  1. Intersect with $\{ab^j c^k\}$,
  2. Consider the homomorphism $h(a) = \epsilon, h(b) = 0, h(c) = 1$.

2 Decision Properties

Emptiness, membership, equivalence

2.1 Conversion among different representations

- ϵ-NFA's to DFA's

  Time complexity $O(n^3 2^n)$ where n^3 comes from the computation for reachable states for all n states. (Assume the size of the alphabet is a constant)

  – Often, the complexity is much smaller.

  Use ***lazy evaluation*** of subsets

$O(n^3 s)$ where s is the number of states in the resulting DFA.

- Automata to RE
  - Recall (worst-case) complexity is high $O(n^3 4^n)$.

    Both the Kleene construction and the state elimination method have the above complexity.
  - Do not convert an NFA to a DFA and then to an RE, as it could be doubly exponential!

2.2 Emptiness of an RL

- Easy to do if we are given an RE explicitly
- If given as an FA, emptiness reduces to graph reachability and has time complexity $O(n^2)$ where n is the number of states.

2.3 Membership

- Testing membership is easy if we are given a DFA. By running the DFA, we can solve it in time linear in $|w|$.
- Testing membership for NFA, ϵ-NFA, and RE

  Time complexity: $O(|w|\, n^2)$.
  - If we are given an NFA, it is more efficient to simulate on an NFA, maintaining the set of states directly.
  - If we are given an ϵ-NFA, compute successor states and the ϵ-closure.
  - If we are given an RE of size n, we can convert it to an ϵ-NFA of size at most $2n$.

3 Equivalence and Minimization

3.1 State equivalence

1. Test if two states of a single DFA are equivalent.

   p, q are equivalent if $\hat{\delta}(p, w)$ is an accept state if and only if $\hat{\delta}(q, w)$ is for all w.

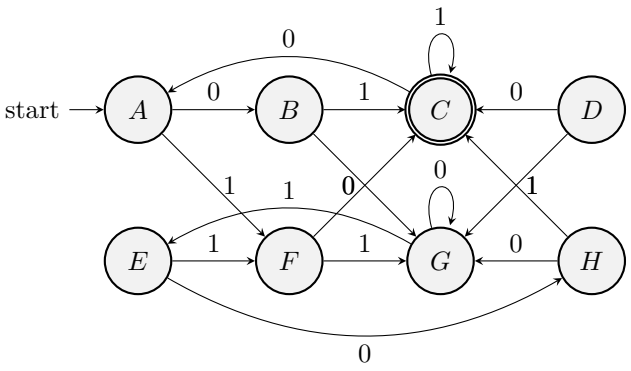
   If two states are not equivalent, we say they are ***distinguishable***.
2. To find equivalent states, we do our best to find pairs of states that are distinguishable.

   States we cannot find distinguishable are equivalent.
3. **Table-filling algorithm**: Recursive discovery of distinguishable pairs

   Basis: If p is accepting and q is non-accepting, they are distinguishable.

   Induction: Let p and q be states such that for some symbol a, $\delta(p, a)$ and $\delta(q, a)$ are a pair of known different states. Then they are distinguishable.

3.2 Example (4.18 of the textbook)



B	x						
C	x	x					
D	x	x	x				
E		x	x	x			
F	x	x	x		x		
G	x	x	x	x	x	x	
H	x		x	x	x	x	x
	A	B	C	D	E	F	G

3.3 Main theorem

> Theorem. If two states are not distinguishable by the table-filling algorithm, then they are equivalent.

Proof. Assume the theorem is false. That is, there are p, q, distinguishable, but the algorithm does not detect that. Call them bad pairs.

Let w be ***one of the shortest strings*** that distinguish bad pairs. Let the bad pair be p, q.

First, w cannot be ϵ by the basis of the algorithm. Write $w = aw'$ and consider $p' = \delta(p, a)$ and $q' = \delta(q, a)$.

1. If p' and q' form a bad pair (distinguishable but not detected), w' will be a shorter (than w) string that distinguishes a bad pair. Contradiction!
2. If p' and q' are distinguishable and detected by the algorithm, the induction part of the algorithm will find p, q distinguishable, and it's a contradiction with the fact that p, q form a bad pair.
3. If p' and q' are equivalent, w cannot distinguish p and q. Contradiction.

As in all possible cases we have a contradiction, our initial assumption is false and the theorem is proved.

□

There are ***two*** important applications of this theorem.

3.4 Test equivalence

- Put two DFAs together and apply the ***table-filling algorithm*** to test if the two start states are equivalent.
- Complexity: Easy to show the $O(n^4)$ bound. We have n^2 pairs, and each will take at most $O(n^2)$ time.

  A careful choice of data structure and analysis shows an $O(n^2)$ bound. For each pair of states, maintain a list of states that depend on it.

3.5 Minimization

1. Remove disconnected parts
2. Partition the states by equivalence.

> Theorem (transitivity). If p, q are equivalent and q, r are equivalent, then p, r are equivalent.

- Easy to show. Proof by contradiction.
- Use the partitions as states.

It is an equivalence relation (reflexivity, symmetry, transitivity).

3.6 Why is it minimal

- Let M be the FA we get from the above. N is an FA with fewer states accepting the same language.
- The starting states of M and N are equivalent.
- For any state p in M, there is a q in N that is equivalent to p. (Consider a string w that brings the automaton M to state p)

  Two states in M are equivalent, a contradiction.